

Project Overview

The project is an internal engineering operations and task management platform designed specifically for your company workflow. The goal is not to clone Jira feature-by-feature, but to build a modern, simplified, engineering-focused platform inspired by the familiarity of Jira's UI while eliminating the operational friction and complexity that currently affects the team.

The platform will focus on:

- simplified workflows,
- fast collaboration,
- realtime communication,
- engineering workflow automation,
- Google Workspace integration,
- GitHub integration,
- client visibility,
- operational transparency.

The system is intended primarily for internal engineering teams, while allowing controlled client participation for project tracking and bug reporting.

Product Philosophy

The architecture and product decisions should follow these principles:

- Minimal configuration
- Opinionated workflows
- Fast UX
- Realtime collaboration
- Developer-first integrations
- Portable projects
- Simpler permissions
- Strong operational visibility

The system should optimize for:

- execution speed,
- communication efficiency,
- visibility across engineering work.

NOT:

- enterprise bureaucracy,
 - over-customization,
 - complex workflow engines.
-

Core Product Modules

1. Authentication & Identity System

Objective

Provide zero-friction secure authentication using company Google Workspace accounts.

Internal Authentication

Internal employees authenticate only via:

- Google OAuth

Restrictions:

- only @nockta.com accounts allowed

Authentication flow:

None

Google Login

↓

Verify company domain

↓

Create/Update user

↓

External Client Access

Clients do not require company accounts.

Recommended approach:

- invite-only access
- magic link authentication
- optional Google OAuth

Client limitations:

- view assigned projects
- report bugs
- comment
- upload files

Clients cannot:

- access internal tasks
 - access internal comments
 - view company analytics
-

Session Strategy

Recommended:

- Access token (short-lived JWT)
 - Refresh token rotation
 - Redis session tracking
-

Authentication Stack

Backend:

- NestJS
- Passport.js
- JWT
- Google OAuth strategy

Infrastructure:

- Redis session store
-

2. Workspace Architecture

Hierarchy

None

Workspace

- ├ Teams
 - ├ Projects
 - ├ Members
 - ├ Integrations
 - └ Settings
-

Workspace Responsibilities

Workspace manages:

- users
 - permissions
 - integrations
 - billing (future)
 - projects
 - activity logs
-

Team System

Teams are lightweight organizational units.

Example:

None

Backend Team

Frontend Team

Mobile Team

DevOps Team

Teams are used for:

- filtering,
- assignments,
- reporting.

NOT permissions inheritance.

3. Permission System

This is one of the most important simplifications compared to Jira.

Workspace Roles

Only:

None

Owner

Admin

Member

Guest

Project Roles

Only:

None

Manager

Contributor

Viewer

Client

Design Philosophy

Avoid:

- nested groups,
- permission schemes,
- workflow permissions,
- inheritance trees.

Permissions should remain:

- understandable,
 - debuggable,
 - predictable.
-

4. Project System

Objectives

Projects should be:

- isolated,
- exportable,

- clonable,
- portable.

This directly solves one of the major Jira pain points.

Project Structure

None

Project

- └ Boards
 - └ Tasks
 - └ Members
 - └ Labels
 - └ Milestones
 - └ Activity Feed
 - └ Integrations
-

Project Features

Supported Views

Board View

Kanban interface.

List View

Spreadsheet/task-table style.

Sprint View

Optional agile organization.

Future Views

- Timeline
 - Calendar
 - Workload
-

5. Task Engine

This is the heart of the platform.

Task Schema

None

Task

- └─ id
- └─ key (ENG-142)
- └─ title
- └─ description
- └─ status
- └─ priority
- └─ labels
- └─ assignee
- └─ reporter
- └─ watchers
- └─ due date
- └─ attachments
- └─ comments
- └─ linked tasks
- └─ linked PRs
- └─ linked commits
- └─ activity timeline
- └─ subtasks
- └─ custom metadata

Task Statuses

Opinionated default workflow:

None
Todo
In Progress
Review
Testing
Done
Blocked

Avoid customizable workflow engines initially.

Task Priorities

None
Low
Medium
High
Critical

Task Relationships

Supported:

- blocks
 - related
 - duplicate
 - parent-child
-

6. Activity Timeline System

This becomes one of the most valuable parts of the platform.

Every significant action becomes an immutable event.

Example Events

None

TaskCreated

TaskAssigned

TaskMoved

CommentAdded

PRLinked

DeploymentFailed

ClientMentioned

Benefits

The timeline provides:

- auditability,
 - operational visibility,
 - historical debugging,
 - collaboration transparency.
-

Recommended Architecture

None

Action

↓

Domain Event



Timeline Event



Notification Pipeline

7. Notification System

The notification system should be event-driven.

NEVER tightly couple notifications to business logic.

Notification Architecture

None

Domain Events



Queue System



Notification Workers



Delivery Channels

Notification Channels

Internal

- In-app notifications
- Google Chat
- Email

Future

- Mobile push
 - Discord
 - Slack
-

Google Chat Integration

One of the most valuable internal features.

Example Notification

None

TASK ASSIGNED

Fix refresh token race condition

Priority: High

Due: Tomorrow

[Open Task]

Future Interactive Features

Potential future actions directly from chat:

- mark task done
 - approve
 - comment
 - assign
-

Recommended Infrastructure

Use:

- Google Chat API
 - Service Account
 - Webhooks
-

8. Realtime Collaboration System

Realtime functionality is critical for operational UX.

Realtime Features

- live board updates
 - live comments
 - typing indicators
 - online presence
 - task movement sync
 - notification streaming
-

Recommended Stack

Backend:

- Socket.IO Gateway in NestJS

Frontend:

- Socket.IO client
-

Realtime Architecture

None
Client Action
↓
API
↓
Database Update
↓
Event Published
↓
WebSocket Broadcast

9. Engineering Workflow Integration

This becomes the platform's strongest long-term advantage.

GitHub Integration

Integrate with:
GitHub

Supported Features

Branch Naming

Suggested:

None
feature/ENG-41-auth-fix

Commit Linking

Commits referencing:

None

ENG-41

automatically attach to tasks.

Pull Request Linking

PRs automatically linked to tasks.

Auto Status Updates

Examples:

None

PR Opened → Review

PR Merged → Testing

Deployment Success → Done

GitHub Webhook Architecture

None

GitHub Webhook

↓

Integration Service

↓

Task Updates

↓
Activity Timeline

↓
Notifications

10. Deployment & CI/CD Tracking

Optional but highly valuable internally.

Supported Integrations

Potential integrations:

- Vercel
 - Railway
 - GitHub Actions
 - Docker pipelines
-

Example Events

None

Deployment Failed

Production Release Succeeded

Rollback Triggered

attached directly to tasks.

11. Client Portal

Clients should have a simplified experience.

Client Capabilities

Clients can:

- view progress,
 - report bugs,
 - upload screenshots,
 - comment,
 - approve tasks.
-

Hidden From Clients

Clients cannot access:

- internal discussions,
 - technical tasks,
 - engineering analytics,
 - sensitive metadata.
-

12. Search System

Fast search is essential.

Search Capabilities

- task titles

- descriptions
 - comments
 - labels
 - assignees
 - project filters
-

Recommended Implementation

Initial:

- PostgreSQL full-text search

Future:

- Elasticsearch/OpenSearch
-

13. File & Attachment System

Supported Uploads

- screenshots
 - logs
 - PDFs
 - videos
 - documents
-

Recommended Storage

Initial

- MinIO (S3 compatible)

Future

- AWS S3 / Cloudflare R2
-

Upload Pipeline

None

Client Upload



Signed Upload URL



Object Storage



Metadata Saved

14. Analytics & Reporting

Internal engineering visibility layer.

Metrics

- overdue tasks
 - sprint velocity
 - workload distribution
 - blocker frequency
 - deployment failures
 - cycle time
-

Dashboard Examples

None

Tasks overdue this week

Blocked tasks

Workload by engineer

Active projects

Deployment health

15. AI Layer (Future)

Since you already have AI infrastructure experience.

Potential future capabilities:

- AI sprint summaries
- standup generation
- duplicate issue detection
- automatic prioritization
- PR summarization
- blocker prediction

Recommended future stack:

- Ollama
- Qdrant
- local LLM inference

NOT part of MVP.

Technical Architecture

Recommended Stack

Backend

NestJS

Reasons:

- modular architecture,
 - DI system,
 - event-driven compatibility,
 - realtime support,
 - scalable structure,
 - ideal for enterprise/internal tooling.
-

ORM

Prisma

Reasons:

- type-safe,
 - migration support,
 - fast development,
 - excellent TypeScript support.
-

Database

PostgreSQL

Reasons:

- relational consistency,
- indexing,
- transactions,
- JSON support,
- reliability.

Cache & Queue

Redis

Use cases:

- queues,
- notifications,
- sessions,
- presence,
- caching.

Queue Processing

BullMQ

Use for:

- notifications,
- emails,
- webhooks,
- retries,
- async processing.

Frontend

React + Vite

Reasons:

- fast iteration,
- internal dashboard suitability,

- simpler than SSR-first solutions.
-

UI Layer

Tailwind CSS + shadcn/ui

Reasons:

- fast UI development,
 - design consistency,
 - Jira-like styling flexibility.
-

State Management

Zustand

Reasons:

- lightweight,
 - simple,
 - scalable enough.
-

Server State

TanStack Query

Reasons:

- caching,
- optimistic updates,
- retries,
- synchronization.

Realtime

Socket.IO

Reasons:

- simple integration,
 - reliable,
 - scalable enough for internal usage.
-

Object Storage

MinIO

Reasons:

- self-hosted,
 - S3 compatible,
 - cost efficient.
-

Infrastructure

Deployment Model

Recommended:

- Dockerized services
 - Docker Compose initially
-

Services

None

frontend

backend

postgres

redis

minio

worker

nginx

Reverse Proxy

Nginx

Responsibilities:

- SSL termination
- routing
- websocket proxying

Monitoring

Recommended:

- Grafana
- Prometheus
- Loki

Track:

- errors
- websocket health
- API latency
- worker failures

Logging Strategy

Use structured logs:

```
JSON
{
  "event": "TASK_ASSIGNED",
  "userId": "123",
  "taskId": "ENG-41"
}
```

Monorepo Structure

```
None
apps/
├─ api
├─ web
└─ workers

packages/
├─ ui
├─ types
├─ sdk
├─ eslint-config
└─ tsconfig
```

API Design

Recommended:

- REST initially

- OpenAPI/Swagger docs

Future:

- GraphQL optional
-

Security Considerations

- JWT rotation
 - RBAC middleware
 - audit logs
 - rate limiting
 - CSRF protection
 - signed upload URLs
 - webhook signature validation
-

UI/UX Direction

The UI should intentionally feel familiar to:
Jira users.

Copy From Jira

- navigation structure
 - board interactions
 - issue modal patterns
 - keyboard shortcuts
 - sidebar layout
-

Avoid From Jira

- excessive admin panels
 - workflow complexity
 - deep configuration trees
 - overloaded issue types
 - complicated permissions
-

Core Strategic Advantage

The platform's real advantage is NOT task CRUD.

The advantage is:

- operational simplicity,
- engineering workflow integration,
- Google Workspace integration,
- realtime collaboration,
- reduced communication overhead,
- internal optimization for your exact engineering culture.